

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2-PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	M. Nivedha	Reg.No.:	192471040
Department:	CS&AS	Date:	

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I M. Nivedha (192471040) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
 I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1	2	3	2	1	2
Q2	2	3	2	0.5	2
Q3	1.5	2	1.5	0.5	1.5
Q4	1.5	2	1.5	0.5	1.5
Q5	2	1.5	2	1	2

RUBRICS FOR CALCULATION:

42

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20% 2	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30% 3	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20% 2	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10% 1	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20% 2	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.

DMA-Based Data Transfer from I/O Device to Memory

Problem Understanding

Design a DMA-based algorithm to transfer a block of data from an I/O device directly to main memory. The CPU should only initialize the DMA controller and handle the completion interrupt. During the actual transfer, the CPU remains available for other tasks.

Inputs

- I/O device
- Starting memory address
- Number of bytes to transfer
- DMA controller
- Interrupt request

Outputs

- Data transferred from I/O device to memory
- DMA completion status
- Interrupt handled successfully

Constraints

- CPU involvement must be minimized.

- DMA must control the data transfer.
- CPU should be interrupted only when necessary.
- Transfer errors must be detected.

Pseudocode

ALGORITHM DMA_Data_Transfer

BEGIN

INPUT Device_ID

INPUT Memory_Address

INPUT Transfer_Size

INITIALIZE DMA_Controller

SET DMA.Source $\leftarrow$ Device_ID

SET DMA.Destination $\leftarrow$ Memory_Address

SET DMA.Count $\leftarrow$ Transfer_Size

SET DMA.Mode $\leftarrow$ DEVICE_TO_MEMORY

SET DMA.Status $\leftarrow$ READY

ENABLE DMA_Interrupt

START DMA_Controller

WHILE DMA.Status = TRANSFERRING DO

 CPU performs other useful tasks

END WHILE

IF DMA.Status = ERROR THEN

 DISPLAY "DMA Transfer Error"

 HANDLE DMA_Error

ELSE

 WAIT FOR DMA_Interrupt

 IF Interrupt_Received = TRUE THEN

 CALL DMA_Interrupt_Handler

 END IF

END IF

END

FUNCTION DMA_Interrupt_Handler

BEGIN

 READ DMA_Status

 IF DMA_Status = TRANSFER_COMPLETE THEN

 CLEAR DMA_Interrupt

 DISPLAY "Data Transfer Completed"

 UPDATE Transfer_Status ← SUCCESS

 ELSE

 UPDATE Transfer_Status ← FAILURE

 END IF

END FUNCTION

Result

The DMA controller transfers data directly from the I/O device to memory while the CPU performs other tasks. The CPU is involved mainly during **DMA initialization and interrupt handling**, thereby reducing CPU overhead and improving system efficiency.

2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.

2. Priority-Based Vectored Interrupt Handling

Problem Understanding

Design an interrupt handling system that receives interrupts from multiple devices. Each device has a priority and a corresponding interrupt vector. The system must select the highest-priority pending interrupt and execute its appropriate service routine.

Inputs

- Multiple interrupt requests
- Device priority
- Interrupt vector
- Service routines

Outputs

- Highest-priority interrupt selected
- Appropriate ISR executed
- Interrupt acknowledged and cleared

Constraints

- Higher-priority devices must be serviced first.
- Each interrupt must map to the correct ISR.
- Pending interrupts must not be lost.
- The system must continue monitoring after servicing.

Pseudocode

ALGORITHM Priority_Vectored_Interrupt_Handler

```

BEGIN

    INITIALIZE Interrupt_Controller
    INITIALIZE Interrupt_Vector_Table

    WHILE System_Running = TRUE DO

        Pending_List ← GET_Pending_Interrupts()

        IF Pending_List is NOT EMPTY THEN

            Highest_Priority ← LOWEST_PRIORITY_VALUE

            FOR EACH Interrupt IN Pending_List DO

                IF Interrupt.Priority > Highest_Priority THEN
                    Highest_Priority ← Interrupt.Priority
                    Selected_Interrupt ← Interrupt
                END IF

            END FOR

            CALL Acknowledge_Interrupt(Selected_Interrupt)

            Vector_Number ← Selected_Interrupt.Vector

            ISR_Address ← Interrupt_Vector_Table[Vector_Number]

            CALL ISR_Address

            CALL Clear_Interrupt(Selected_Interrupt)

        ELSE

            CPU performs normal processing

        END IF

    END WHILE

END

```

```
FUNCTION Acknowledge_Interrupt(Interrupt)
```

```
BEGIN
```

```
    SEND Acknowledge signal to Interrupt_Controller
```

```
END FUNCTION
```

```
FUNCTION Clear_Interrupt(Interrupt)
```

```
BEGIN
```

```
    CLEAR Interrupt.Request
```

```
END FUNCTION
```

Result

The algorithm continuously monitors interrupt requests, selects the **highest-priority pending interrupt**, obtains its vector address, and dispatches the corresponding service routine. This provides efficient and organized interrupt management.

3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.

Dynamic Selection of Memory-Mapped I/O and I/O-Mapped I/O

Problem Understanding

Design an algorithm that selects between memory-mapped I/O and I/O-mapped I/O according to the application's latency and bandwidth requirements.

Inputs

- Latency requirement
- Bandwidth requirement
- Address-space availability
- Device characteristics

Outputs

- Selected I/O method
- Configuration of the selected method

Constraints

- Low latency should be prioritized for performance-sensitive devices.
- High bandwidth requirements should favor efficient memory-style transfers.
- Address-space limitations should be considered.
- The selected method must satisfy device requirements.

Pseudocode

```
ALGORITHM Select_IO_Mapping
```

```
BEGIN
```

INPUT Required_Latency
INPUT Required_Bandwidth
INPUT Available_Address_Space
INPUT Device_Type

SET MMIO_Score $\leftarrow$ 0
SET PMIO_Score $\leftarrow$ 0

IF Required_Latency = LOW THEN
 MMIO_Score $\leftarrow$ MMIO_Score + 2
ELSE
 PMIO_Score $\leftarrow$ PMIO_Score + 1
END IF

IF Required_Bandwidth = HIGH THEN
 MMIO_Score $\leftarrow$ MMIO_Score + 2
ELSE
 PMIO_Score $\leftarrow$ PMIO_Score + 1
END IF

IF Available_Address_Space = SUFFICIENT THEN
 MMIO_Score $\leftarrow$ MMIO_Score + 1
ELSE
 PMIO_Score $\leftarrow$ PMIO_Score + 2
END IF

IF MMIO_Score > PMIO_Score THEN

 Selected_Mode $\leftarrow$ MEMORY_MAPPED_IO

ELSE IF PMIO_Score > MMIO_Score THEN

 Selected_Mode $\leftarrow$ IO_MAPPED_IO

ELSE

 IF Device_Type = HIGH_PERFORMANCE THEN
 Selected_Mode $\leftarrow$ MEMORY_MAPPED_IO
 ELSE
 Selected_Mode $\leftarrow$ IO_MAPPED_IO
 END IF

END IF

CONFIGURE Device USING Selected_Mode

DISPLAY "Selected I/O Mode:", Selected_Mode

END

Result

The algorithm dynamically evaluates latency, bandwidth, address-space availability, and device requirements before selecting the appropriate I/O mapping technique. This allows the system to adapt the I/O mechanism to application requirements.

4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.

4. Bus Arbitration for PCIe, USB, and Thunderbolt

Problem Understanding

Design a bus arbitration algorithm that manages multiple devices requesting communication over PCIe, USB, and Thunderbolt interfaces. The arbiter should consider priority, bandwidth requirements, fairness, and interface characteristics.

Inputs

- Device requests
- Interface type
- Device priority
- Bandwidth requirement
- Current bus status

Outputs

- Selected device
- Granted communication access
- Updated priority/fairness information

Constraints

- High-priority requests should receive faster service.
- Lower-priority devices must not be permanently blocked.
- Bandwidth requirements must be considered.
- Only one eligible transaction should be granted at a time within the arbitration domain.

Pseudocode

ALGORITHM Multi_Interface_Bus_Arbitration

BEGIN

 INITIALIZE Device_Queue

 INITIALIZE Priority_Table

 INITIALIZE Fairness_Counter

```

WHILE System_Running = TRUE DO

    Request_List ← GET_Device_Requests()

    IF Request_List is EMPTY THEN
        CONTINUE
    END IF

    FOR EACH Request IN Request_List DO

        IF Request.Interface = PCIe THEN
            Request.Score ← Request.Priority + 3
        ELSE IF Request.Interface = Thunderbolt THEN
            Request.Score ← Request.Priority + 2
        ELSE IF Request.Interface = USB THEN
            Request.Score ← Request.Priority + 1
        END IF

        IF Request.Bandwidth = HIGH THEN
            Request.Score ← Request.Score + 2
        END IF

        IF Request.Wait_Time > WAIT_THRESHOLD THEN
            Request.Score ← Request.Score + FAIRNESS_BONUS
        END IF

    END FOR

    Selected_Request ← Request_With_Highest_Score(Request_List)

    IF Bus_Available(Selected_Request.Interface) = TRUE THEN

        GRANT_BUS(Selected_Request.Device)

        PERFORM Data_Transfer(Selected_Request)

        RELEASE_BUS(Selected_Request.Interface)

        UPDATE Fairness_Counter
        REMOVE Selected_Request FROM Request_List

    ELSE

```

```

        INCREASE Selected_Request.Wait_Time

    END IF

END WHILE

END

```

```

FUNCTION Bus_Available(Interface)

```

```

BEGIN

```

```

    IF Interface is FREE THEN
        RETURN TRUE
    ELSE
        RETURN FALSE
    END IF

```

```

END FUNCTION

```

Result

The arbitration algorithm evaluates pending requests using **priority, interface type, bandwidth requirement, and waiting time**. It grants access to an appropriate device while adding a fairness mechanism to prevent starvation.

5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

5. Hybrid I/O: Polling for Low-Speed and DMA + Interrupts for High-Speed Devices

Problem Understanding

Design a hybrid I/O mechanism that chooses polling for low-speed devices and DMA with interrupts for high-speed devices. This reduces unnecessary interrupt overhead for slow devices while allowing efficient high-speed data transfer.

Inputs

- Device speed
- Data size
- Latency requirement
- Device type

Outputs

- Selected I/O method
- Completed data transfer
- Transfer status

Constraints

- Low-speed devices should use polling.
- High-speed devices should use DMA.
- DMA completion should generate an interrupt.
- The CPU should avoid unnecessary waiting for high-speed transfers.

Pseudocode

ALGORITHM Hybrid_IO_Communication

BEGIN

INPUT Device_ID

INPUT Device_Speed

INPUT Data_Size

INPUT Latency_Requirement

IF Device_Speed = LOW THEN

CALL Polling_Mode(Device_ID)

ELSE IF Device_Speed = HIGH THEN

CALL DMA_Interrupt_Mode(Device_ID, Data_Size)

ELSE

IF Data_Size <= SMALL_DATA_LIMIT THEN

CALL Polling_Mode(Device_ID)

ELSE

CALL DMA_Interrupt_Mode(Device_ID, Data_Size)

END IF

END IF

END

FUNCTION Polling_Mode(Device_ID)

BEGIN

DISPLAY "Polling Mode Selected"

WHILE Data_Available(Device_ID) = FALSE DO

CPU checks device status

```

END WHILE

READ Data_From_Device(Device_ID)

DISPLAY "Polling Transfer Completed"

END

FUNCTION DMA_Interrupt_Mode(Device_ID, Data_Size)

BEGIN

    DISPLAY "DMA Mode Selected"

    CONFIGURE DMA.Source ← Device_ID
    CONFIGURE DMA.Count ← Data_Size
    CONFIGURE DMA.Mode ← DEVICE_TRANSFER

    ENABLE DMA_Interrupt

    START DMA

    WHILE DMA.Status = TRANSFERRING DO
        CPU performs other useful tasks
    END WHILE

    WAIT FOR DMA_Interrupt

    IF DMA.Status = TRANSFER_COMPLETE THEN
        CLEAR DMA_Interrupt
        DISPLAY "DMA Transfer Completed"
    ELSE
        DISPLAY "DMA Transfer Failed"
    END IF

END FUNCTION

```

Result

The hybrid mechanism uses **polling for low-speed devices** and **DMA with interrupts for high-speed devices**. This minimizes CPU overhead while maintaining efficient communication and appropriate response times.

Final Conclusion

These pseudocode designs show how **DMA, interrupts, I/O mapping, bus arbitration, and hybrid communication** can be combined to improve computer-system performance. DMA reduces CPU workload during large transfers, vectored interrupts provide rapid response to devices, dynamic I/O selection adapts to performance requirements, bus arbitration manages competing devices fairly, and hybrid I/O chooses the most suitable mechanism according to device speed. Therefore, the proposed algorithms improve **CPU utilization, throughput, latency, and overall I/O efficiency**.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs

Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient